

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

第5章

セキュリティを意識した
Vibe Codingの勘所

開発環境とユーザーの安全性、守るべき2つの軸を意識しよう

Vibe Coding時代の開発者には、従来以上に厳格な、「開発環境を守る責任」と「ユーザーを守る責任」の2つの責任が生まれます。本章では、Vibe Coding時代に開発者が担う責任について、この2つを軸に解説し、安全な実践方法を紹介します。

Author Kyohei Fukuda

Twitter @labelmake

URL https://www.youtube.com/@kyohei_devVibe Codingが変えた
2つのセキュリティ責任

Vibe Codingは開発効率を大幅に向上させる一方で、AIへの過度な依存による開発環境からの情報漏洩や、脆弱性の埋め込みといった新たなセキュリティリスクをもたらします。従来の開発では、エンジニアが1行1行コードを書き、その意味を理解していました。しかしVibe Codingでは、自然言語でAIに指示を出し、AIがコードを生成します。AIエージェントは開発者と同じファイルシステムやネットワーク権限を持つため、AIが悪意あるプロンプトインジェクションに応じたり、悪意あるMCPサーバと連携したりすると、機密情報漏洩に直結します。また、AIが生成したコードをユーザーが直接使用することになるので、もしコードに脆弱性が含まれていたら、即座にセキュリティインシデントにつながります。これらの特性から、Vibe Coding時代の開発者には従来以上に厳格な、「開発環境を守る責任」と「ユーザーを守る責任」の2つの責任が生まれます。

AIエージェントを介した情報の外部送信や、悪意のあるMCPサーバによる情報窃取などは、開発環境における機密情報や認証情報の漏洩につながります。AIエージェントに与える権限を適切に制限するなどの対策によって、開発環

境を守る必要があります。これは自分自身はもちろん、一緒に作業するチームメンバー、会社や顧客を守るということでもあります。

AIが生成するコードは「動いた」だけで満足せずに、セキュリティ要件を満たしているかを検証し、脆弱性によるリスクからユーザーを守る必要があります。この責任自体は、Vibe Codingで登場した新しいものではありません。法的にも倫理的にも、ずっと開発者はユーザーの安全を守る義務を負ってきました。しかし、AIに開発のハンドルを握らせるVibe Codingでは、事故を起こさない保証を人間が担うということ、あらためて強く認識する必要があります。

この章では、Vibe Coding時代に開発者が担う責任について、この2つを軸に解説し、安全な実践方法を紹介します。

開発環境を守るための
セキュリティ

Vibe Codingでは、AIエージェントが開発者と同じ権限でファイルシステムやネットワークにアクセスできるため、開発環境そのものが攻撃対象になります。ここでは、実際の事例を紹介しながら、実行環境における2つの主要リスクと、それに対する防御の実践方法を解説します。

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

実行環境の2つのリスク

■ 環境変数のコンテキスト混入による機密情報流出

Vibe Codingでは、プロンプトインジェクションなどの攻撃によってコンテキストウィンドウに含まれる環境変数や設定ファイルが悪用される危険があります。プロンプトインジェクションとは、AIに対する悪意のある指示を、正規の入力データに紛れ込ませる攻撃手法です。AIの発展とともに、ここ数年で次第に大きな脅威になってきています。

実際に起こった事例として、2025年6月に発見され、8月に修正された「GitHub Copilot CamoLeak脆弱性 (CVSS 9.6) ^{注1)}」があります。攻撃者はGitHubのpull request (以下PR)を作成し、ユーザーには見えないMarkdownのコメントに「隠しプロンプト」を埋め込みます。すると、ユーザーがGitHub CopilotにこのPRをレビューさせた際、隠しプロンプトの指示に基づいて、環境変数を定義する.envファイルなどの機密情報の内容を抽出させることができました。盗まれたデータは、正規のGitHub CDN プロキシ(「Camo」と呼ばれています)を経由する画像URLとして外部に流出し、従来のネットワーク出口制御を迂回しました。

このようなAIエージェントによる情報流出への対策としては、使用するコーディングエージェントの設定で機密ファイルへのアクセスを制限するのが有効です。Claude Codeでは、settings.jsonのpermissions.denyを設定することで、特定のコマンドや操作をClaude Codeから実行できないようにさせる権限ルールを指定できます。たとえば、Read(./env)を指定すると、環境変数を読み込む操作が拒否されるようになります。重要なのは、どの製品を使う場合でも、AIエージェントが機密情報にアクセ

スできないように設定するという考え方です。

■ MCPサーバ経由の認証情報窃取

導入したMCPサーバを経由して、認証情報が窃取されるケースも懸念されています。

たとえば、あるセキュリティ研究チームが、サプライチェーン攻撃の概念実証として開発した「DevTools-Assistant」は、表向きはプロジェクト分析ツールですが、実態は.envやSSH鍵、クラウド認証情報を窃取し、正規のGitHub APIトラフィックに偽装して外部送信するマルウェアです^{注2)}。この実験により、MCPサーバを悪用したサプライチェーン攻撃が現実的な脅威であることが実証されました。

MCPサーバはAIエージェントと同じ権限で動作し、AIエージェントの提供元が第三者サーバを監査することはありません。AIのハルシネーションで提案される架空のパッケージを、攻撃者が先回りして登録するスロップスクワッティング(Slopsquatting)という攻撃も発生しています。

MCPサーバを導入する際は、まず「公式性」を確認することが重要です。開発元が信頼できるか、ダウンロード数・GitHubスター数・コントリビューター数などを確認し、実績があるMCPサーバを選ぶべきです。そのうえで、コーディングエージェントの設定から、機密情報(SSH鍵、クラウド認証情報、環境変数)へのアクセスをブロックし、重要な操作には承認を必須化することで、権限を最小化します。

サンドボックスで守る開発環境

上記2つのリスクに共通するのは、機密情報が「読み取られ」、その後「外部に送信される」という2段階の攻撃パターンです。CamoLeak脆弱性では.envファイルが読み取られてGitHub CDN経由で外部送信され、DevTools-

注1) <https://www.legitsecurity.com/blog/camoleak-critical-github-copilot-vulnerability-leaks-private-source-code>

注2) <https://securelist.com/model-context-protocol-for-ai-integration-abused-in-supply-chain-attacks/117473/>

Assistantでは認証情報が窃取されてGitHub APIトラフィックに偽装して外部に流出しました。つまり、これらの攻撃を防ぐためには、ファイルシステムへのアクセス制限とネットワークアクセスの制御が必要となります。

上述のとおり、これらはAIエージェントの権限管理によっても制限できます。しかし、このような権限設定は人為的なミスや設定漏れが起こり得るソフトウェアレベルの制御です。サンドボックスはOSレベルでファイルシステムとネットワークアクセスを強制的に分離するため、たとえ承認プロンプトに間違えて回答したとしても、攻撃の影響を最小化できます。

■ サンドボックスのしくみ

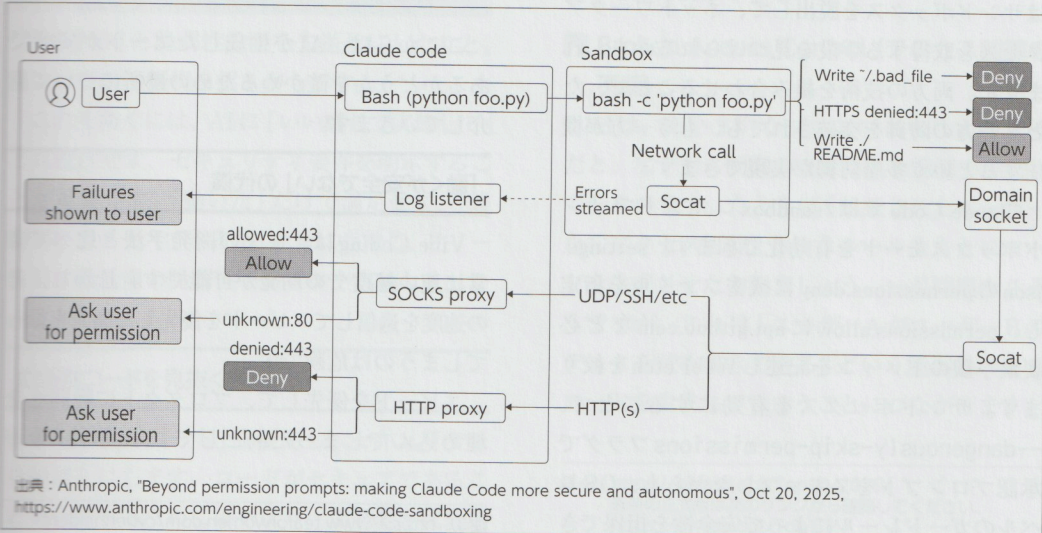
サンドボックスとは、プログラムの実行環境を隔離し、システムへのアクセスを制限する技術です。サンドボックス内のプロセスは、許可されたディレクトリやネットワークにのみアクセスでき、それ以外へのアクセスはOSレベルでブロックされます。たとえばAIエージェントがBashコマンドを実行する際、そのコマンドはサンドボックス内で実行されます。これにより、たとえAIエージェントがプロンプトイ

ンジェクションで悪意のあるコマンドを実行しようとしても、物理的にアクセスが不可能となります。Claude CodeにはSandboxing機能が搭載されており、OSレベルのサンドボックス機能を活用できます(図1)。macOSではSeatbelt、LinuxではbubblewrapというOS標準の隔離機構を使用して、AIエージェントが実行するプロセスを制限された環境内に閉じ込めます。OpenAIのCodex CLIなどにも同様のサンドボックス技術が提供されています。

ファイルシステム分離では、AIエージェントは現在の作業ディレクトリへの読み書きのみが許可され、それ以外のファイル修正はブロックされます。これにより、SSH鍵やクラウド認証情報などの機密システムファイルへの不正アクセスが防止され、プロンプトインジェクションを受けた場合でもシステムファイルを改変されることがありません。

ネットワーク分離では、AIエージェントは承認されたサーバへの接続のみが許可されます。インターネットアクセスはサンドボックス外で動作するプロキシサーバ経由でのみ可能となり、このプロキシサーバがドメイン許可リスト(allowlist)に基づいてアクセスを制限します。

▼図1 Claude Codeのサンドボックスアーキテクチャ



Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

許可リストに含まれないドメインへのアクセスが発生した場合はユーザーに確認を要求するため、プロンプトインジェクションを受けた場合でも、攻撃者のサーバへの機密情報送信やマルウェアのダウンロードを防ぐことができます。

このドメイン許可リストの運用が重要です。たとえばgithub.com全体を許可すると、Camo LeakのようにGitHub CDNプロキシを悪用したデータ流出が可能になってしまいます。開発に必要な最小限のドメイン(api.github.com、registry.npmjs.orgなど)のみを許可し、それ以外はブロックするか都度確認する設定が推奨されます。

セキュリティの観点からは少し外れた話題になりますが、AIエージェントが誤ってrm -rf ~/のような破壊的コマンドを実行し、ホームディレクトリ全体を削除してしまう事故も複数報告されています。サンドボックスによるファイルシステム分離は、このような誤操作の影響を作業ディレクトリ内に封じ込めます。

効果的なサンドボックスには、ファイルシステムとネットワークの両方の分離が必要です。ネットワーク分離がなければ、侵害されたエージェントはSSH鍵などの機密ファイルを外部に送信できてしまいます。一方、ファイルシステム分離がなければ、侵害されたエージェントはサンドボックスを脱出して、ネットワークアクセスを取得する手段を見つけられるかもしれません。両方の技術を組み合わせることで、たとえ一方の防御を突破されても、もう一方が機能するという多層防御が実現できます。

Claude Codeでは/sandboxコマンドでサンドボックスモードを有効化できます。settings.jsonのpermissions.denyに機密ファイルを指定し、permissions.allowにapi.github.comなど必要最小限のドメインを記述しWebFetchを絞ります。サンドボックスを有効にすることで、--dangerously-skip-permissionsフラグで承認プロンプトをスキップしながらも、OSレベルのガードレールによって安全性を担保でき

ます。サンドボックスなしでこのフラグを使用することは、AIエージェントに無制限のアクセス権を与えることになり、セキュリティ上推奨されません。

■ サンドボックスでは防げない課題

サンドボックスは機密ファイルへのアクセスや許可されていないネットワーク通信を防ぎますが、「許可された範囲内」での問題には対処できません。たとえば、間接プロンプトインジェクションなどにより、作業ディレクトリ内のソースコードにバックドアを仕込んだり脆弱性を埋め込んだりする攻撃に対しては、ファイルへのアクセスが許可されている以上、サンドボックスでは防げません。

そのため、AIが生成したコード自体のセキュリティは、CI/CDに統合したSAST(静的解析)ツールやコードレビューで担保する必要があります。また、開発用のクラウド認証情報には本番環境へのアクセス権限を持たせないなど、サンドボックス以外の部分でも最小権限の原則を徹底することが重要です。サンドボックスは多層防御の一部として位置づけ、これだけに頼らないようにしましょう。

ユーザーを守るためのセキュリティ



ここからは、AIが生成したコードが安全であるかどうかを確かめるための勘所について紹介していきます。

「動くが安全でない」の代償

Vibe Codingは、従来の開発手法と比べて非常に速い速度での開発が可能です。しかし、その速度を過信してスピードを優先した開発を行ってしまうのは危険です。

スピードを優先して、プロダクトに脆弱性を埋め込んでしまった例として「Tea事件^{注3)}」が

注3) <https://www.teaforwomen.com/cyberincident>

あります。この事件自体はVibe Codingによる開発ではないと言われていますが、「動いたからOK」というスピード優先の開発姿勢が招く典型的な脆弱性パターンとして教訓になります。

2025年7月、アメリカの出会い系アプリ「Tea」から、約72,000件の画像（うち約13,000件がセルフイー画像と身分証明書を含む）が流出しました。原因は、データを保存していたFirebase Storageの設定がallow read: if true;（全員が読み取り可能）となっていたためです。つまり、攻撃者が全ファイルに直接アクセスできる状態になっていたということです。

この事例は、Backend as a Service (BaaS) 設定ミスの典型例とも言えます。Firebaseと同様のBaaSであるSupabaseにも、RLS (Row Level Security) などの危険な設定があります。この設定が無効になっていると、SQLにおけるSELECT、INSERT、UPDATE、DELETEのすべての操作が可能で、誰でもすべてのデータにアクセス・操作できる状態になってしまいます。

この事件の本質は、「動いたからOK」という楽観的な開発姿勢にあります。Vibe Codingの「まず動かす」文化は、異常系や権限チェックを軽視して、最も順調に動作するハッピーパスのみを確認したり、AIへの過信から生成コードの内部ロジックを理解しないまま進めたり、「後でやる」と脆弱性診断を先送りしたりと、セキュリティ軽視を招きがちです。

これを防ぐには、AIに「いい感じで実装して」と丸投げせず、セキュリティ要件を明示することが重要です。「動いた」だけで満足せず、「誰が何にアクセスできるか」を常に意識し、リリース前に最低限の脆弱性チェックを実施しましょう。

危険なコードを見抜く視点

AIは「動くコード」を優先し、セキュリティを後回しにします。コードがセキュアであることを保証するためにはやはり、開発者が代表的

な脆弱性パターンを理解し、生成物を適切にレビューすることが重要です。以下に、OWASP Top 10に挙げられている脆弱性パターンを基に、Vibe Codingにおける危険なコードとその対処を紹介します。なお、本章で紹介する項目は、最新のOWASP Top 10 2025 RC (Release Candidate) を参照しています^{注4}。

Insecure Design (安全が確認されない不安な設計)

Vibe Codingでは「いい感じで実装して」というあいまいなプロンプトで開発が進むため、設計段階でのセキュリティ考慮が抜け落ちます。安全でない設計を避ける第一歩は、不要な個人情報を持たないことです。必要最小限のデータのみを収集・保持・処理するという考え方、「データ最小化の原則」に基づいて設計しましょう。

個人情報を取り扱うことに対する責任を甘く見積もってはいけません。次のように、AIにプロンプトを投げる前に、「このデータは本当に必要か？」と自問しましょう。

- ・本当にフルネーム、生年月日、住所が必要か？ ニックネーム、年齢層（「20-29」などの粗い粒度）で十分ではないか？
- ・クレジットカード情報は外部サービス（Stripe など）に委譲できないか？

Broken Access Control (アクセス制御の不備)

AIは、あいまいなプロンプトを与えただけだと、アクセス制御のロジックを憶測で作成してしまいます。たとえば「認証されたユーザーならアクセス可能」というルールをもとにコードを生成させたとしましょう。一見問題なさそうですが、これは「ユーザーAがユーザーBのデータを読み書きできる」という脆弱性のあるコードを生成してしまうことにつながります。

注4) <https://owasp.org/Top10/2025/>
最新の正式版は次のリンクから確認してください。
<https://owasp.org/Top10/>

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

AIへの指示は要件をクリアにし、アクセス制御のロジックはできれば複数人でレビューしましょう。たとえばシェアする必要のないデータであれば、Supabase RLSなら`auth.uid()` = `user_id`、Firebaseなら`request.auth.uid` == `userId`を検証し、ユーザーIDベースの権限制御を徹底します。

Injection (インジェクション)

AIは文字列連結でクエリを構築しがちです。もし`SELECT * FROM users WHERE name = '${name}'`のようなクエリが生成されてしまったら、攻撃者が`name = '' OR '1'='1'`を入力すると全ユーザー情報が漏洩します。

こういったSQLをはじめとするインジェクションへの対策としては、ORMやプリペアドステートメントを使用することでリスクを低減できます。Reactの`dangerouslySetInnerHTML`やVueの`v-html`を使う場合は、`DOMPurify`でサニタイズします。LLMの出力も「信頼できない入力」として扱い、必ずサニタイズします。

依存関係の監視

AIは、目的の生成物に必要な依存関係を自動提案するため、野良のライブラリを平気で開発環境に持ち込みます。気づかぬうちにサプライチェーン攻撃の標的にされているかもしれない、ということを意識しましょう。

たとえば、2025年9月のnpmパッケージ乗っ取り事件では、週3.6億ダウンロードの`debug`、週3億ダウンロードの`chalk`を含む18パッケージが、開発者アカウント乗っ取りにより悪意のあるバージョンを公開されました。公開期間はわずか2時間でしたが、CI/CDで自動更新するプロジェクトは即座に影響を受けました。AIの盲点として、AIは学習後に発見されたCVE(脆弱性)、ハッキング手法を考慮できません。AI生成コードで特定バージョンのライブラリを使っていたところ、後にそのバージョンに深刻なCVEが判明したケースもあります。

実践的対策としては、まずDependabot、OSV、Trivyなどの診断ツールをCI/CDに統合し、脆弱性を継続的にスキャンすることが基本です。これらのツールは、依存ライブラリの既知の脆弱性(CVE)を自動検出し、修正版へのアップデートを提案してくれます。人手では追跡しきれない数千のライブラリの脆弱性を継続的に監視できるため、必須の対策と言えます。

次に、新しいライブラリを導入する前には、選定基準を設けてチェックすることが重要です。過去3ヵ月以内の更新があるか、コントリビューター数は十分か、週1万ダウンロード以上か、などの基準をチームですり合わせておくとう安心です。

さらに、悪意のあるリリースは通常、発見後の数時間以内にレジストリから削除されるため、リリース直後のパッケージを避ける、インストール前にスキャンしてリスクを回避する、などの対策もあります。たとえばNode.jsのパッケージマネージャーであるpnpmでは、`.npmrc`で`minimum-release-age`を設定できます。1日の遅延などを設定すれば、上記のような2時間の間だけ公開されたリリースによる攻撃などを回避可能です。現在npmでは使えない設定ですが、今後導入されることを期待しています。また、Socket.devはリアルタイムでnpmパッケージの挙動を監視し、悪意のあるコード(データ流出、シェルコマンド実行など)を検出可能です。BunであればSecurity Scanner APIにてパッケージインストール時にセキュリティチェックを挟み込むことができます。

「監督者」としての基礎力

AIがコードを書く量が増すほど、開発者の役割は「コーダー」から「レビューし、品質を保証する監督者」へと変化します。AIの生成物は過度に信頼せず、ユーザー入力と同じく「信頼できないデータ」として扱うマインドセットが重要です。

とはいえ、Vibe CodingではAIの出力すべ

てを人間がチェックするのは現実的ではありません。そこで重要なのが、ツールによる自動化と人間のレビューの適切な役割分担です。

CI/CDでは、SAST (静的解析)、依存関係スキャン、Linter、テストを自動実行し、機械的にチェック可能な脆弱性を網羅的に検出します。SonarQube^{注5}やGitHub CodeQL^{注6}などのSASTツールはSQLインジェクションの典型的なパターン、既知のCVEを持つライブラリの使用、コーディング規約違反などを高速に検出します。第3章で解説されるCI/CD統合により、AIの出力は自動的にこれらのチェックを通過することになります。

加えて、Claude Codeを使用している場合は`/security-review`コマンドも活用できます。このコマンドは、従来のパターンマッチングベースの静的解析とは異なり、コードのセマンティクス (意味) を理解したうえで脆弱性を検出します。SQLインジェクション、XSS (クロスサイトスクリプティング)、認証／認可の欠陥などを検出し、詳細な説明と修正提案を提供してくれるため、セキュリティレビューの効率が大幅に向上します。GitHub Actionsとの統合も可能で、PRの作成時に自動でセキュリティレビューを実行できます^{注7}。

しかし、ツールの出力、成果物を判断し、適切なアクションを起こすのは人間です。ツールの誤検知を見抜き、ツールが検出できない論理的な脆弱性に気づくためには、セキュリティの基礎知識を持った人間によるレビューが不可欠です。人間のレビューは「ツールでは検出できない領域」に集中すべきです。

人間がレビューすべき重要ポイントとしては次のようなものがあります。

- ・アーキテクチャ設計：信頼境界の設計、機密情報の保存／利用場所、暗号化の適用範囲

など、システム全体を俯瞰^{ふくかん}した判断

- ・アクセス制御ロジック：「誰が何にアクセスできるか」というビジネス要件が正しく実装されているかの検証
- ・外部サービスとの連携：BaaSのセキュリティルール、APIキーのスコープなど、設定ミスがインシデントに直結する領域
- ・新しい依存関係：CVEチェックを通過しても、メンテナンス状況やコミュニティの信頼性は人間が判断

これらに集中することで、すべてのコードを詳細にレビューせずとも、セキュリティ上重要な部分を効率的に確認できます。

基礎を身につけるために

上記のレビューを行うには、セキュリティの原理原則を理解した基礎知識が不可欠です。ここで言う「基礎」とは、表面的なツールの使い方ではなく、「なぜその脆弱性が危険なのか」「なぜその対策が有効なのか」を理解できる力を指します。たとえば、認証と認可の違いを理解していなければ、AIが生成したアクセス制御のロジックが安全かを判断できません。このライブラリは使ってよいのでしょうか？ サブライチェーン攻撃のリスクを理解していないと判断できません。静的解析ツールの警告は誤検知でしょうか？ 脆弱性の原理を知らないと判断できません。

学習の出発点としては、OWASP Top 10 (最新版：2025 RC) で典型的脆弱性を学ぶことから始めるのがよいでしょう。それぞれの脆弱性の概要、防止方法、攻撃シナリオを把握することができます。『体系的に学ぶ 安全なWebアプリケーションの作り方 第2版^{注8}』も、脆弱性の原理から対策まで実践的に解説されており入門におすすめです。

前述したnpmパッケージ乗っ取り事件、Camo

注5) <https://www.sonarsource.com/jp/>

注6) <https://codeql.github.com>

注7) <https://github.com/anthropics/claude-code-security-review>

注8) 徳丸 浩 著、SBクリエイティブ、2018年

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

Leak脆弱性など、セキュリティの脅威は日々進歩します。そのため、知識の継続的アップデートも重要です。BleepingComputer^{注9}や、上記で紹介したSocket.dev^{注10}やWiz^{注11}などのセキュリティ製品の開発会社のブログなどを定期的にチェックし、最新脅威に対応しましょう。筆者のX (@labelmake^{注12}) やYouTubeチャンネル^{注13}でも、セキュリティ関連のトピックを定期的に取り上げていますので、併せてご参照ください。

Vibe Coding以外にも共通する本質ですが、ツールが進化すればするほど、それを使いこなす人間の基礎力が重要になります。ツールの進化を真に活用するには、徹底的な基礎がなけれ

ば振り回されてしまいます。ツールに使われるのではなく、ツールを使いこなすために、セキュリティの原理原則を理解しましょう。

まとめ

Vibe Coding時代において、開発者は「コーダー」から「監督者」へと役割を転換する必要があります。これは2つの責任……開発者自身を守ることと、エンドユーザーを守ることを意味します。AIの出力を常に批判的に評価する姿勢を保つこと、生成されたコードは必ずレビューすること、そして何より、ツールの進化を真に活用するには徹底的な基礎力が不可欠であるという認識を持つこと、このマインドセット転換が、安全なVibe Codingの鍵です。ツールに使われるのではなく、ツールを使いこなしましょう。SD

注9) <https://www.bleepingcomputer.com>

注10) <https://socket.dev/blog>

注11) <https://www.wiz.io/blog>

注12) <https://x.com/labelmake>

注13) https://www.youtube.com/@kyohei_dev

技術評論社



齊藤慎仁、
株式会社クラウドネイティブ 著
A5判 / 304ページ
定価(本体2,700円+税)
ISBN 978-4-297-14970-3

大好評
発売中!

ITセキュリティ・ゼロトラスト概論

——本質を理解し現代的なITインフラを理解するために

現代の組織は急速な変化に直面しており、リモートワーク普及、DX推進、AI活用が進む一方で、情報漏洩やセキュリティ事故も多発しています。このような背景から「ゼロトラスト」が注目されていますが、多くの企業が具体的な対応に迷い、不適切な導入により効果を得られていない現状があります。本書は、読者が自社のITインフラを主体的に設計できるようになることを目指し、ゼロトラストの概念と具体的な実装の両方を解説します。筆者の幅広い経験に基づき、形式的なHow toではなく、「何をやるべきか(What)」と「なぜそれを行うのか(Why)」を明確にすることで、各企業が自社の課題解決にゼロトラストを活かせるようになることを目的としています。

こんな方に
おすすめ

- ・ ITセキュリティ担当者
- ・ ゼロトラスト導入を検討中の方